

Royal Estate

Full-stack Real Estate Listing Platform

Submission Report

Project Report

A production-ready real estate marketplace with listings, search & filters, authentication, image uploads (Cloudinary), multilingual UI (RTL/LTR), and messaging/inbox module.

React (Vite)

Tailwind CSS

Redux Toolkit

Node.js + Express

MongoDB (Mongoose)

Cloudinary

KEY MODULES

Home Highlights • Search • Listing Details • Profile • Create/Update Listings • Inbox

FOCUS

Fast UX, clean architecture, secure auth, scalable roadmap for Dubai Verified Contributor submission.

Royal Estate App

Generated: Jun 2026

Royal Estate App — Project Report (Submission)

Full-stack

React (Vite)

Node/Express

MongoDB

Cloudinary

Project Type: Full-stack Real Estate Listing Platform

Frontend: React (Vite) + Tailwind CSS + Redux Toolkit + i18n

Backend: Node.js + Express + MongoDB (Mongoose)

Media: Cloudinary (image upload & delivery)

How the app works (Property + Listing + ROI)

1) Guest experience (no login)

- Open `/` (Home): frontend calls `GET /api/listing/home` to load featured offers, recent rent, and recent sale listings quickly.
- Click any property card: opens `/listing/:listingId` and renders full listing details (images from Cloudinary).
- Use header search: navigates to `/search?searchTerm=...` and fetches filtered lists.

2) Sign up / Sign in (authorization)

- Sign in sets a cookie named `access_token` (JWT, HTTP-only).
- Backend middleware `verifyToken` reads this cookie and attaches `req.user`.`
- Protected pages/components work only when the token is valid (Profile, Create/Update Listing, Inbox).

3) How you list (create) a property (Owner flow)

- User fills Create Listing form (title, description, address, beds/baths, rent/sale, offer/discount, etc.).
- Upload images to Cloudinary:
 - Client sends image to backend upload route.
 - Backend uploads to Cloudinary and returns an image URL.
- Submit listing: backend runs `POST /api/listing/create` (requires token) and stores listing with `userRef = req.user.id`.
- Search/Home automatically shows the new listing after it's stored (home uses the fast home endpoint).

4) Update / Delete (security)

- Backend checks ownership:
 - `POST /api/listing/update/:id` updates only if `req.user.id === listing.userRef`.

- DELETE /api/listing/delete/:id deletes only if req.user.id === listing.userRef.

5) Messaging / Inbox

- Send message: POST /api/message/send (requires token)
- Get inbox: GET /api/message/inbox (requires token)
- Messages can be linked to a listing via listingId, and sender info is populated.

6) Where “ROI” comes in (what this enables for users)

- Fast browsing and search increases conversion (more views -> more enquiries).
- Owner-only controls reduce mistakes and protect data integrity.
- Cloudinary image delivery improves visual quality, which typically improves click-through.

1) Executive Summary

Royal Estate App is a production-ready property marketplace where users can browse, search, and view listings, while authenticated users can create and manage their own property listings. The platform supports multilingual UI (including RTL), image uploads, authentication, and a modern responsive interface.

2) Core Features (Functionalities)

A) Public (Guest) Features

- **Home page highlights**
 - Featured offers (carousel/slider)
 - Recent rent listings
 - Recent sale listings
- **Search & filtering**
 - Filter by rent/sale, offer, furnished, parking
 - Keyword search (name/address/description)
 - Sorting (newest/price etc. — based on API parameters)
- **Listing details page**
 - Full listing information
 - Images gallery (Cloudinary URLs)
 - Pricing and offer/discount display

- **About page**

B) Authentication & User Accounts

- Sign up / Sign in
- JWT-based session handling (token + cookies)
- Protected routes:
 - Profile
 - Create/Update listing
 - Inbox (messaging)

C) Listings (CRUD)

- Create listing:
 - Title, description, address
 - Beds/baths, furnished, parking
 - Rent/Sale, offer/discount
 - Multiple images upload
- Update listing
- Delete listing
- Owner-only permissions

D) Media Uploads

- Cloudinary image upload pipeline
- CDN delivery + optimized transformations

E) Messaging / Inbox

- Send messages
- Conversation handling
- Inbox UI (protected)

F) Internationalization (i18n)

- Multi-language UI
- RTL support (Arabic direction switching)
- Dynamic language switch behavior

3) System Architecture (High Level)

Frontend (Client)

- React SPA using Vite bundler
- React Router for navigation
- Redux Toolkit + redux-persist for state persistence
- API communication via Axios instance

- Swiper for hero carousel
- Cloudinary images optimized with srcset and responsive sizes

Backend (API)

- Express REST endpoints:
 - /api/auth/* authentication
 - /api/user/* profile/user endpoints
 - /api/listing/* listings endpoints (search, CRUD)
 - /api/upload/* uploads to Cloudinary
 - /api/message/* messaging endpoints
 - MongoDB persistence via Mongoose models
 - CORS configured for local + deployed origins
-

4) Current Tech Stack (As Implemented)

Frontend

- React (Vite)
- Tailwind CSS
- Redux Toolkit, react-redux, redux-persist
- react-router-dom
- axios
- i18next, react-i18next
- Swiper
- react-helmet-async

Backend

- Node.js, Express
- MongoDB, Mongoose
- jsonwebtoken (JWT)
- cookie-parser
- cors
- multer + Cloudinary storage
- dotenv

Deployment (Typical)

- Frontend: Vercel / similar static hosting
 - Backend: Render / similar Node hosting
 - Database: MongoDB Atlas
 - Media: Cloudinary
-

5) Performance & UX Optimizations Included

- Responsive images (Cloudinary URL transforms + `srcset`), lazy loading for non-critical images
- Homepage data strategy: dedicated fast endpoint for home sections (offers/rent/sale), avoids expensive empty-regex scans

6) Security & Reliability

- JWT authentication
- Protected routes for private pages
- Owner-only authorization for listing updates/deletes
- Central error handling middleware (backend)
- Credentialed CORS for secure cookie usage

7) Future Stack / Roadmap (Recommended)

A) Frontend (Speed + SEO)

- Next.js (SSR/SSG/ISR) for faster first load + SEO
- Route-based code splitting + lazy loading heavy components
- Advanced image optimization via Cloudinary presets / Next Image
- PWA for offline caching and instant repeat loads

B) Backend (Scale + Latency)

- Redis caching for homepage and popular searches
- Rate limiting + request validation (zod/joi)
- Observability: OpenTelemetry tracing + structured logs
- Queue system (BullMQ/Redis) for background jobs

C) Search (Better relevance)

- MongoDB Atlas Search or Elasticsearch/Meilisearch
- Autocomplete + typo tolerance

D) Infrastructure / DevOps

- Docker + CI/CD (GitHub Actions)
- CDN + edge caching (Cloudflare)
- Environment separation (dev/staging/prod)

E) Product Features

- Favorites / wishlist
- Agent / broker accounts
- Appointment scheduling
- Payments/subscriptions for premium listings
- Admin dashboard (moderation, analytics)

- Maps integration + nearby places
-

8) Test Plan (Quick Checklist)

- Verify home sections load under a few seconds
 - Search & filters return correct results
 - Auth flows (signup/signin/signout)
 - Create/update/delete listing permissions
 - Image uploads work and images render on listing pages
 - Inbox messaging basic flow
 - Language switch + RTL layout checks
-

9) Conclusion

Royal Estate App is a complete full-stack real estate platform with strong core marketplace features (listings + search + auth + media + messaging) and a clear roadmap for enterprise-grade performance and scalability.